Golden Thread Plugin

Golden Thread Plugin — Documentation

Version gt 0.6.0 / gt-wiki 0.1.0

Golden Thread turns an Obsidian vault into the single source of truth for all AI memory across every project and every session. v0.6.0 adds a tiered rule enforcement model with hook-backed Core rules. gt-wiki 0.1.0 adds an LLM-powered knowledge base with immutable sources and interlinked pages.

What It Does

Instead of scattered `.claude/memory/` files and CLAUDE.md snippets that live and die per session, Golden Thread gives every fact a permanent home in a structured vault. Knowledge flows up a hierarchy from session notes into the vault, and the right facts are always in scope when you need them.

```
session conversation
      ↓  gt-work / gt-promote
project memory  (memory/*.md)
      ↓  gt-promote
project files   (research.md / decisions.md / design.md / spec.md / runbook.md)
      ↓  gt-promote
Knowledge/      (cross-project wiki pages, backed by immutable Sources/)
      ↓  gt-promote
global-memory/  (loaded in every session, all projects)
```

Facts move up the hierarchy as they prove themselves general. They never move back down and are never silently deleted.

Core Rules (gt 0.6.0)

Golden Thread defines a tiered rule model that separates rules by scope and enforcement strength.

Three scope levels:

Level	Meaning
Core	Applies to every chat, every project, no exceptions. Un-removable.
Context	Applies to this project or session type only.
Generic	Default; no special standing.

Two enforcement strengths:

Strength	Meaning
Reminder	Injected on every turn via `UserPromptSubmit` hook.
Validated	Checked by a `Stop` hook that blocks any reply violating it; machine-enforced.

Core/Validated rules are hook-backed. Hook scripts install to `~/.claude/golden-thread/hooks/` during `install.sh`. `settings.json` references them by absolute path so they survive vault renames and project moves. `gt_paths.py` is a self-healing resolver — if a recorded path is stale, it locates `core-rules/` by scanning the vault rather than failing silently.

The canonical rule definitions live in `Projects/golden-thread/core-rules/` inside the vault. Editing a rule file there changes what the hook injects — the rule text is never duplicated into the script.

The canary: `core_timestamp_every_message.md` is designated Core/Validated not because a missing timestamp causes real harm, but because it is trivially observable at the top of every reply. When the timestamp disappears, enforcement has broken.

gt Skills (11)

Setup

Command	What it does
`/gt:gt-init`	Create a new vault from scratch, or wire an existing vault to a project. Idempotent — safe to re-run.
`/gt:gt-create`	Scaffold a new project folder with the standard structure. Captures your brain dump into `idea.md`. Supports sub-projects, tags, and runbook creation.

Daily Work

Command	What it does
`/gt:gt-open`	Load a project at session start. Reads all project docs in order (idea → research → decisions → design → spec → runbook → memory), summarizes state, and asks where to pick up.
`/gt:gt-work`	Write back session findings. Appends to `research.md`, adds ADRs to `decisions.md`, refines `design.md`, creates `spec.md` when design is complete, and flags content for PROTOCOL.md.
`/gt:gt-ingest`	Bulk-import an existing project's memory files, CLAUDE.md rules, and notes into the vault. External sources are stored immutably in `Sources/` before being synthesized into Knowledge pages.
`/gt:gt-review`	Scan recent Obsidian daily notes for uncaptured tasks and ideas. Surfaces them

Command	What it does
	grouped by date, then promotes selected ones into tracked project folders.

Knowledge Management

Command	What it does
`/gt:gt-query`	Look something up — reads `index.md` first, follows wikilinks, falls back to grep. The vault is always checked before the web.
`/gt:gt-promote`	Graduate a fact up the hierarchy: project memory → project files → Knowledge wiki page → global-memory. Also handles new project scaffolding and retiring stale content.
`/gt:gt-refresh`	Check `Sources/` for upstream changes. Supersedes outdated sources with new immutable files — never edits the old one. Updates Knowledge pages that cited the changed source.

Maintenance

Command	What it does
`/gt:gt-lint`	Audit the vault for structural problems: broken wikilinks, orphaned pages, missing index entries, unlisted memory files, Knowledge pages citing superseded sources, and stale pages.
`/gt:gt-runbook-lint`	Scan all project `runbook.md` files for content that has drifted into multiple runbooks. Routes duplicated content to the right shared layer via `gt-promote`.

gt-wiki Skills (5)

The gt-wiki plugin provides an LLM-powered knowledge base separate from the Golden Thread memory hierarchy. Sources are ingested immutably; Knowledge pages are synthesized summaries linked bidirectionally.

Key design decision: The vault `CLAUDE.md` is kept slim (architecture + pointer only). Page format schema lives in `Knowledge/_template.md` and is read on demand by the ingest and refresh skills — it is not loaded into every session's context.

Command	What it does
`/gt:gt-wiki-init`	Set up a new wiki vault from scratch. Runs `vault_init.py` deterministically — creates `Sources/`, `Knowledge/`, `index.md`, `log.md`, seeds `CLAUDE.md` from template, and stamps `Knowledge/_template.md`.
`/gt:gt-wiki`	Query the wiki. Reads `index.md` first, follows wikilinks, falls back to grep, deep-digs Sources for precision. Logs every query.
`/gt:gt-wiki-ingest`	Add a source: fetch or paste, store immutably in `Sources/`, discuss with user, write Knowledge pages per `_template.md`, cross-link bidirectionally, update index and log.
`/gt:gt-wiki-lint`	Run `wiki_lint.py` (10 deterministic checks). Interprets findings, proposes fixes, records declines in `lint-declines.md` so nothing gets re-litigated.
`/gt:gt-wiki-refresh`	Check selected sources for upstream changes. Supersedes changed ones with new immutable source files. Updates citing Knowledge pages.

Vault Structure

```
<vault>/
  CLAUDE.md                  ← architecture + pointer to Knowledge/_template.md
  index.md                   ← navigational index of all Knowledge pages
  log.md                     ← audit trail: every create, ingest, promote, retire
  review-queue.md            ← items flagged for owner review (written by gt-lint)

  Sources/                   ← IMMUTABLE raw originals
    YYYY-MM-DD <title>.md

  Knowledge/                 ← cross-project wiki pages
    _template.md             ← page format schema (read by ingest/refresh skills)
    <Page Title>.md

  global-memory/
    MEMORY.md                ← index; loaded every session
    <topic>.md

  Projects/
    README.md                ← master project list
    CONVENTIONS.md           ← lifecycle phases, file roles, tag taxonomy
    PROTOCOL.md              ← cross-project process rules
    INFRASTRUCTURE.md        ← server fleet, defined once

    <project-slug>/
      README.md              ← status board + YAML frontmatter
      source.md              ← where code lives, topology, deploy file plan
      idea.md                ← original brain dump — IMMUTABLE
      research.md            ← append-only findings
      decisions.md           ← append-only ADRs
      design.md              ← current architecture
      spec.md                ← handoff artifact (created when design is complete)
      runbook.md             ← operational procedures (optional)
      memory/
        MEMORY.md            ← session memory index
        <topic>.md

    golden-thread/
      core-rules/            ← canonical Core rule definitions
```

Immutability Model

File	Mutability	Rule
`Sources/*.md`	Immutable	Never edited; superseded by new files
`idea.md`	Immutable	Origin story; never changed
`research.md`	Append-only	History is extended, never rewritten
`decisions.md`	Append-only	Reversed by adding a new ADR, never by editing
`design.md`	Mutable	Always describes the current architecture
`spec.md`	Mutable (scope only)	Self-contained handoff doc
`runbook.md`	Mutable	Operational HOW-TO; incubator before CLAUDE.md
`PROTOCOL.md`	Append-only	Cross-project process rules
`Knowledge/*.md`	Mutable	Synthesized summaries; cite `Sources/`
`core-rules/*.md`	Mutable	Editing here changes what hooks inject

Context Footprint

What loads	When	Approx tokens
`~/.claude/CLAUDE.md` (global instructions)	Every session	~200
`<vault>/CLAUDE.md` (architecture + pointer)	Every session in vault dir	~100
`global-memory/MEMORY.md` (index only)	Every session	~50
`Knowledge/_template.md` (page schema)	Only during ingest/refresh	~150
Project memory files	On demand via `gt-open`	Varies

The slim CLAUDE.md design (introduced in gt-wiki 0.1.0) drops the vault's cold-start footprint from ~1,300 tokens to ~350.

Source Topology

Every project records where its code lives in `source.md`.

Topology	Meaning
`local`	A folder on this machine. No deploy step.
`remote`	One repo, one server; server pulls from GitHub.
`bastion-jump`	Several servers, all reached through one gateway host.
`bastion-direct`	Several servers, each reachable independently.

Bastion projects carry a **file plan** marking each deployed file `static` (byte-identical everywhere) or `unique` (per-server variant that must never be cross-deployed).

Install

```
# From zip
unzip golden-thread-plugin.zip
cd golden-thread-plugin
bash install.sh
# Restart Claude Code
```

Installs both `gt` (v0.6.0) and `gt-wiki` (v0.1.0) as separate plugins under the `golden-thread-plugin` marketplace. Requires Python 3.8+.

Typical Session Pattern

```
/gt:gt-open my-project        # load project, summarize state

# ... work happens ...

/gt:gt-work                   # write back findings, update docs

# periodically
/gt:gt-lint                   # catch structural drift
/gt:gt-wiki-ingest <url>      # add a source to the wiki
/gt:gt-promote                # graduate a finding to Knowledge or global-memory
```

Requirements

- Python 3.8+
- Claude Code (any version)
- Obsidian (optional — the vault is plain markdown; Obsidian is the GUI layer)